

说明

本应用笔记介绍如何使用 OpenOCD 与 GDB 对 WCH RISC-V 系列 MCU 进行在线调试。OpenOCD 在调试体系中充当调试服务器的角色，负责与硬件调试器通信并管理目标芯片；GDB 作为调试客户端，通过 TCP 端口与 OpenOCD 建立连接，发送调试指令并接收目标状态信息。

适用范围

适用范围	系列
通用 MCU	CH32L103 等

目录

说明	1
目录	2
表格索引	2
图片索引	错误!未定义书签。
第 1 章 调试环境搭建	1
1.1 硬件准备	1
1.1.1 软件工具链	1
第 2 章 调试流程	2
2.1 启动 openocd	2
2.1.1 参数逐项说明	2
2.1.2 运行示例	2
2.2 启动 GDB	3
2.3 调试时重新加载符号以及重新加载程序	3
历史版本	5
声明	6

表格索引

调试软件列表	1
常用 GDB 命令	3

第1章 调试环境搭建

1.1 硬件准备

使用 OpenOCD + GDB 进行调试需要以下硬件：

- 目标 MCU：WCH RISC-V 系列 MCU 开发板或目标板
- 调试器：WCH-Link
- 连接线：根据目标芯片调试接口

WCH-Link 是一款多功能在线调试和下载模块，支持 WCH RISC-V 架构 MCU 以及 ARM 内核 MCU 的在线调试和下载，同时带有一路串口输出功能。

1.1.1 软件工具链

所需软件工具如下：

表 1-1 调试软件列表

工具	说明	获取方式
OpenOCD (WCH 版)	调试服务器，支持 WCH-Link 系列调试器	MounRiver Studio/MounRiver StudioII 的安装目录下
RISC-V GDB	调试客户端，支持 RISC-V 架构	同上
wch-riscv.cfg	WCH RISC-V 目标配置文件	同上
终端工具	用于运行 OpenOCD 和 GDB 命令	同上

第2章 调试流程

2.1 启动 openocd

典型的启动方式如下

```
openocd.exe ^
-c "gdb_port 3333" ^
-c "tcl_port 6666" ^
-c "telnet_port 4444" ^
-s "<OPENOCD_SCRIPTS_PATH>" ^
-f "<OPENOCD_SCRIPTS_PATH>/wch-riscv.cfg" ^
-c "chip_id CH32L10x"
```

命令由六部分组成，依次完成端口配置 => 脚本搜索路径设置 => 加载目标配置文件 => 指定芯片型号四个阶段。

2.1.1 参数逐项说明

```
-c "gdb_port 3333"
```

指定 OpenOCD 的 GDB Server 监听端口为 3333。GDB 将通过 target extended-remote :3333 连接此端口，建立远程调试会话。

```
-c "tcl_port 6666"
```

指定 TCL 脚本接口端口为 6666。该端口用于外部工具通过 TCL 脚本控制 OpenOCD，MounRiver Studio 的调试前端即通过此端口与 OpenOCD 交互。

```
-c "telnet_port 4444"
```

指定 Telnet 命令接口端口为 4444。开发者可通过 telnet localhost 4444 连接，直接向 OpenOCD 发送原生命令（如 reset halt、flash erase_sector、mdw 等），适合底层调试和手工验证。

```
-s "<OPENOCD_SCRIPTS_PATH>"
```

设置 OpenOCD 的脚本搜索路径。MounRiver Studio 将其指向 OpenOCD 安装目录下的 bin 文件夹，使 OpenOCD 能够定位到 target/、interface/ 等子目录中的配置文件。

```
-f "<OPENOCD_SCRIPTS_PATH>/wch-riscv.cfg"
```

加载 WCH 官方目标配置文件 wch-riscv.cfg。该文件定义了 CH32 RISC-V 系列的目标参数

```
-c "chip_id CH32L10x"
```

在加载配置文件后，通过 -c 传入芯片型号标识。该参数告知 OpenOCD 目标芯片的具体型号为 CH32L10x（CH32L103 系列），使 OpenOCD 能够选择正确的 Flash 算法和内存映射。该参数和 MRS 内下载配置界面的芯片型号参数一致。

2.1.2 运行示例

在执行开启 openocd 后会输出如下内容，表示此时 openocd 服务进程已经启动，开始监听端口

```
Info : only one transport option; autoselect 'sdi'
Warn : Transport "sdi" was already selected
Ready for Remote Connections
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : WCH-LinkE mode:RV version 2.22
Info : wlink_init ok
Info : clock speed 6000 kHz
Info : [wch_riscv.cpu.0] datacount=2 progbufsize=8
```

```
Info : [wch_riscv.cpu.0] Examined RISC-V core; found 1 harts
Info : [wch_riscv.cpu.0] XLEN=32, misa=0x40901105
[wch_riscv.cpu.0] Target successfully examined.
Info : starting gdb server for wch_riscv.cpu.0 on 3333
Info : Listening on port 3333 for gdb connections
```

2.2 启动 GDB

打开另一个终端，启动 RISC-V GDB，例如：

```
riscv32-wch-elf-gdb.exe <FIRMWARE.ELF>
```

启动时<FIRMWARE.ELF>是可选项，如未指定 elf，后续若需要符号解析，可以在 GDB 中执行 `file <FIRMWARE.ELF>` 来加载符号

随后连接 openocd，执行

```
target remote localhost:3333
```

localhost:3333 是启动 openocd 时指定的监听端口

连接后程序会暂停，此时可以执行

```
monitor reset halt
```

来复位程序。

后续便可通过 GDB 的命令来控制芯片行为，一下是常用的 GDB 指令的汇总

表 2-1 常用 GDB 命令

命令	缩写	功能
break <location>	b	在指定位置设置断点，绝对地址打断点需要在地址前加上*
continue	c	继续运行至断点或中断
step	s	单步执行，进入函数
next	n	单步执行，跳过函数
stepi	si	单步执行一条汇编指令
info registers	i r	显示所有寄存器
backtrace	bt	显示调用栈
info locals	—	显示当前局部变量
x/<n><fmt> <addr>	—	检查内存内容
print <expr>	p	打印表达式值

2.3 调试时重新加载符号以及重新加载程序

上文中提到可以通过在 GDB 中调用 `file <FIRMWARE.ELF>` 来重新加载 GDB 的符号。但是实际并没有同步程序到芯片。若要在加载符号的同时下载程序，可在 GDB 中执行：

```
load <FIRMWARE.ELF>
```

此时 GDB 会输出

```
Loading section .startup, size 0xf6 lma 0x0
Loading section .vectors, size 0x114 lma 0xf8
Loading section .text, size 0xda3a lma 0x20c
Loading section .rodata, size 0xd84 lma 0xdc48
Loading section .data, size 0xd0 lma 0xe9cc
Loading section .noncacheable, size 0x1500 lma 0xea9c
Start address 0x00000000, load size 65432
Transfer rate: 12 KB/sec, 7270 bytes/write.
```

可以看到，在加载符号的同时下载了程序。

除了使用 GDB 调试外，还可以使用其他脚本语言通过 tcl 或者 telnet 的端口调试，实现自动化调试或者批量验证。

历史版本

日期	版本	变更内容
2026/7/8	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。